

# Détecter des faux billets avec Python

Yannis TALLY  
Data Analyst chez Orange Business

Mission 2026 – pour l'ONCFM



# Contexte du projet

## Contexte de l'entreprise

L'ONCFM ( l'Organisation nationale de lutte contre le faux-monnayage) met en place des méthodes d'identification des faux billets

## Enjeux

Détecter rapidement et efficacement les faux billets après scan physique des billets (longueur, hauteur, largeur, etc.)

Mettre à disposition des équipes une application fiable de prédiction.

## Objectifs

Analyser un jeu de données de 1 500 billets scannés.

Tester et comparer 4 algorithmes de Machine Learning

# Les étapes et outils pour mener à bien le projet

## Etapes :

Chargement & Exploration des Données

Préparation des Données

Modélisation — Test des 4 Algorithmes

Comparaison & Choix du Modèle

Application Fonctionnelle

## Outils :



# Chargement & Exploration des Données

1500 billets (Faux / Vrais)

7 colonnes (descriptions)

Détection des valeurs manquantes

Les valeurs manquantes de `margin_low` seront traitées par régression linéaire afin de conserver un maximum d'information plutôt que de supprimer ces lignes.

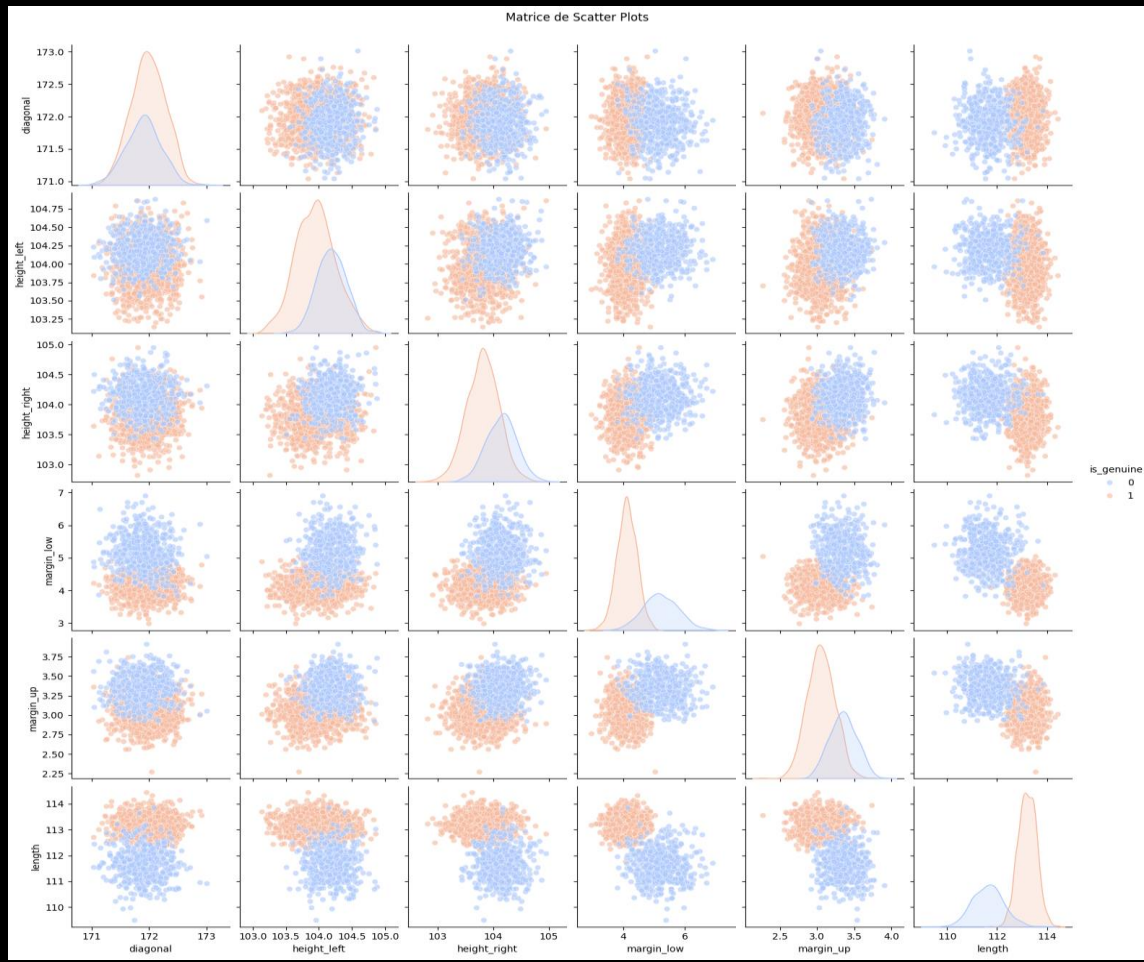
|      | is_genuine | diagonal | height_left | height_right | margin_low | margin_up | length |
|------|------------|----------|-------------|--------------|------------|-----------|--------|
| 0    | True       | 171.81   | 104.86      | 104.95       | 4.52       | 2.89      | 112.83 |
| 1    | True       | 171.46   | 103.36      | 103.66       | 3.77       | 2.99      | 113.09 |
| 2    | True       | 172.69   | 104.48      | 103.50       | 4.40       | 2.94      | 113.16 |
| 3    | True       | 171.36   | 103.91      | 103.94       | 3.62       | 3.01      | 113.51 |
| 4    | True       | 171.73   | 104.28      | 103.46       | 4.04       | 3.48      | 112.54 |
| ...  | ...        | ...      | ...         | ...          | ...        | ...       | ...    |
| 1495 | False      | 171.75   | 104.38      | 104.17       | 4.42       | 3.09      | 111.28 |
| 1496 | False      | 172.19   | 104.63      | 104.44       | 5.27       | 3.37      | 110.97 |
| 1497 | False      | 171.80   | 104.01      | 104.12       | 5.51       | 3.36      | 111.95 |
| 1498 | False      | 172.06   | 104.28      | 104.06       | 5.17       | 3.46      | 112.25 |
| 1499 | False      | 171.47   | 104.15      | 103.82       | 4.63       | 3.37      | 112.07 |

1500 rows × 7 columns

# Chargement & Exploration des Données

## Distribution des Variables

Les variables `margin_low` et `length` sont les plus discriminantes entre vrais et faux billets. Elles seront probablement les variables les plus importantes dans nos modèles de Machine Learning.



# Prétraitement

## Localisation des valeurs manquantes

### Gestion de ces dernières :

La régression linéaire permet d'estimer les valeurs manquantes de `margin_low` en s'appuyant sur les autres variables corrélées. Cette méthode est plus précise que la simple imputation par la moyenne car elle respecte la structure des données.

### Vérification

```
(variable) df: DataFrame
print(df.isnull().sum())
```

|              |    |
|--------------|----|
| is_genuine   | 0  |
| diagonal     | 0  |
| height_left  | 0  |
| height_right | 0  |
| margin_low   | 37 |
| margin_up    | 0  |
| length       | 0  |
| dtype: int64 |    |

```
print(df_imputed.isnull().sum())
```

|              |   |
|--------------|---|
| is_genuine   | 0 |
| diagonal     | 0 |
| height_left  | 0 |
| height_right | 0 |
| margin_low   | 0 |
| margin_up    | 0 |
| length       | 0 |
| dtype: int64 |   |

```
from sklearn.linear_model import LinearRegression
import numpy as np

def impute_with_linear_regression(df, target_col):

    # Colonnes utilisées comme features (toutes sauf la cible)
    feature_cols = [col for col in df.columns if col != target_col]

    # Lignes sans NaN → pour entraîner le modèle
    df_complete = df.dropna()

    # Lignes avec NaN dans la colonne cible → à imputer
    df_missing = df[df[target_col].isna()]

    # Séparation X / y sur les données complètes
    X_train = df_complete[feature_cols]
    y_train = df_complete[target_col]

    # Entraînement du modèle
    lr = LinearRegression()
    lr.fit(X_train, y_train)

    # Prédiction des valeurs manquantes
    X_missing = df_missing[feature_cols]
    valeurs_predites = lr.predict(X_missing)

    # Remplacement des NaN
    df_imputed = df.copy()
    df_imputed.loc[df[target_col].isna(), target_col] = valeurs_predites

    return df_imputed

# Application
df_imputed = impute_with_linear_regression(df, target_col="margin_low")
```

# Prétraitement

## Séparation des variables explicatives de la cible

### Entraînement sur une partie des données et test sur une autre:

La séparation 80/20 est un standard en Machine Learning. Elle garantit que les modèles sont évalués sur des données jamais vues pendant l'entraînement, ce qui mesure leur capacité à généraliser sur de nouveaux billets.

Le K-means, étant non supervisé, n'utilisera pas cette séparation. Il sera entraîné et évalué sur l'ensemble des 1 500 billets.

## Standardisation des données

```
X = df_imputed[["diagonal","height_left","height_right","margin_low","margin_up","length"]] # Toutes les colonnes sauf la cible
y = df_imputed["is_genuine"] # La colonne cible
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42, test_size=0.2)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
# Use scaler on X_train and X_test.

colonnes = X_train.columns
X_train = scaler.fit_transform(X_train)

X_train = pd.DataFrame(X_train, columns=colonnes)

#On fait pas le fit sur le test car le fit permet à la machine d'apprendre les statistiques ce qu'on veut pour les entraînements mais pas pour le test
#Rappel le but est que la machine se teste sur de nouvelles données non apprises

X_test = scaler.transform(X_test)

X_test = pd.DataFrame(X_test, columns=colonnes)
```

# Modélisation - Paramètres

Random\_state = 42 pourquoi ?

Max\_iter = 100 pourquoi ?

| Paramètre     | Valeurs possibles                                                   | Description                                                                                       |
|---------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| penalty       | 'l1', 'l2', 'elasticnet', None                                      | Type de <b>régularisation</b> appliquée                                                           |
| C             | float (défaut: 1.0)                                                 | <b>Inverse de la force de régularisation</b> — plus C est petit, plus la régularisation est forte |
| solver        | 'lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga' | <b>Algorithme d'optimisation</b> utilisé                                                          |
| max_iter      | int (défaut: 100)                                                   | Nombre maximum d' <b>itérations</b> pour la convergence                                           |
| multi_class   | 'auto', 'ovr', 'multinomial'                                        | Stratégie pour la <b>classification multi-classes</b>                                             |
| class_weight  | None, 'balanced', dict                                              | Gestion du <b>déséquilibre de classes</b>                                                         |
| fit_intercept | True, False                                                         | Inclure ou non un <b>biais (intercept)</b>                                                        |
| tol           | float (défaut: 1e-4)                                                | <b>Tolérance</b> pour le critère d'arrêt                                                          |
| l1_ratio      | float entre 0 et 1                                                  | Ratio L1/L2 pour <b>ElasticNet</b> uniquement                                                     |
| random_state  | int, None                                                           | Graine aléatoire pour la <b>reproductibilité</b>                                                  |
| n_jobs        | int, None                                                           | Nombre de <b>cœurs CPU</b> utilisés                                                               |

```
from sklearn.linear_model import LogisticRegression

# 1) Initialisez une régression logistique.
# Quels sont les paramètres ajustables de ce modèle ?
# Bien que vous puissiez laisser la majorité d'entre eux par défaut,
# il vaut mieux, à minima, ajuster 'max_iter' pour plafonner le temps de calcul.

# 2) Entraînez le modèle avec vos données X_train et y_train

model = LogisticRegression(random_state=42, max_iter=100)

model.fit(X_train, y_train)

# 3) Utiliser la régression logistique pour déterminer les vrais / faux billets de X_test.

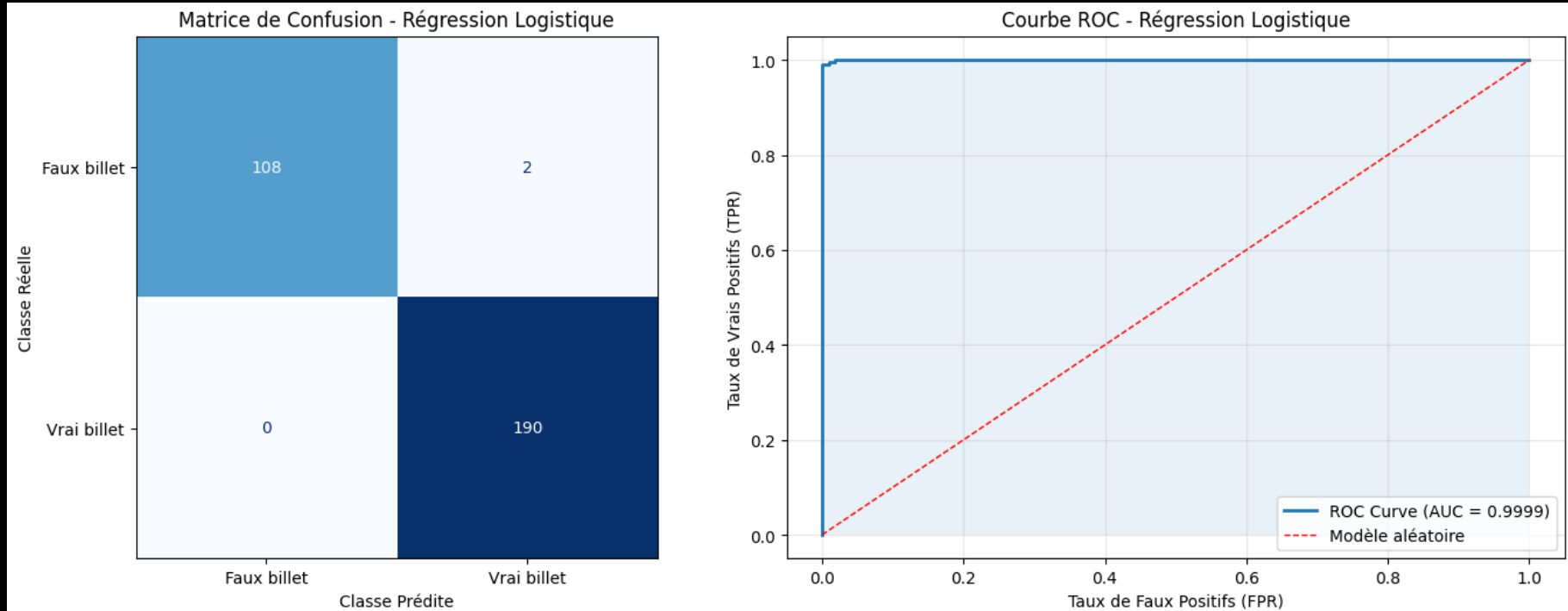
y_pred = model.predict(X_test)

y_proba = model.predict_proba(X_test)
```



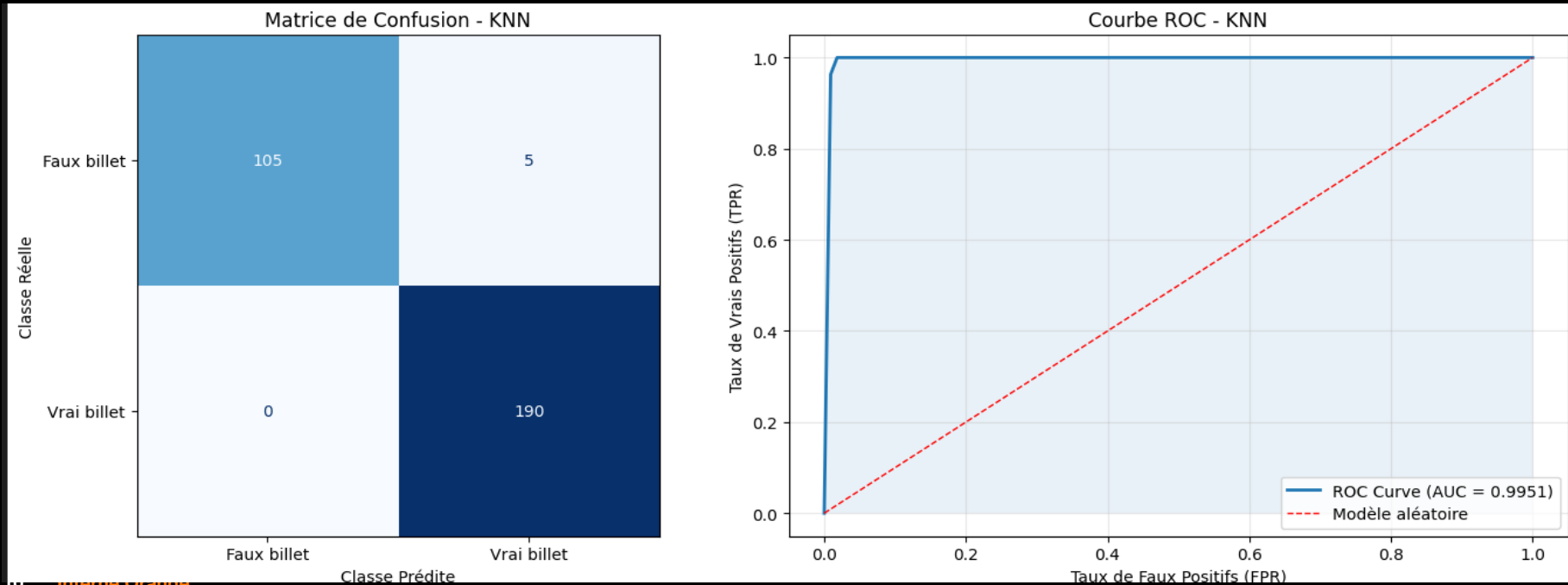
# Modélisation – Régression Logistique

Que pouvons nous en déduire ?



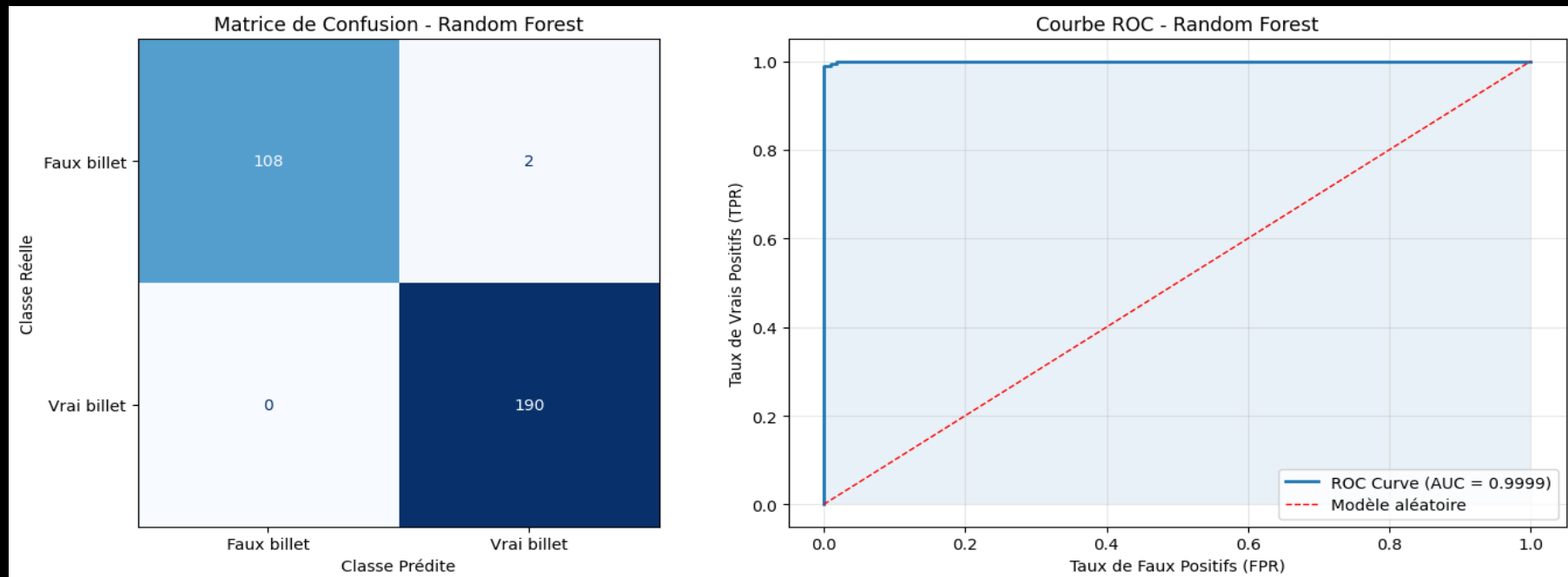
# Modélisation – KNN

Que pouvons nous en déduire ?



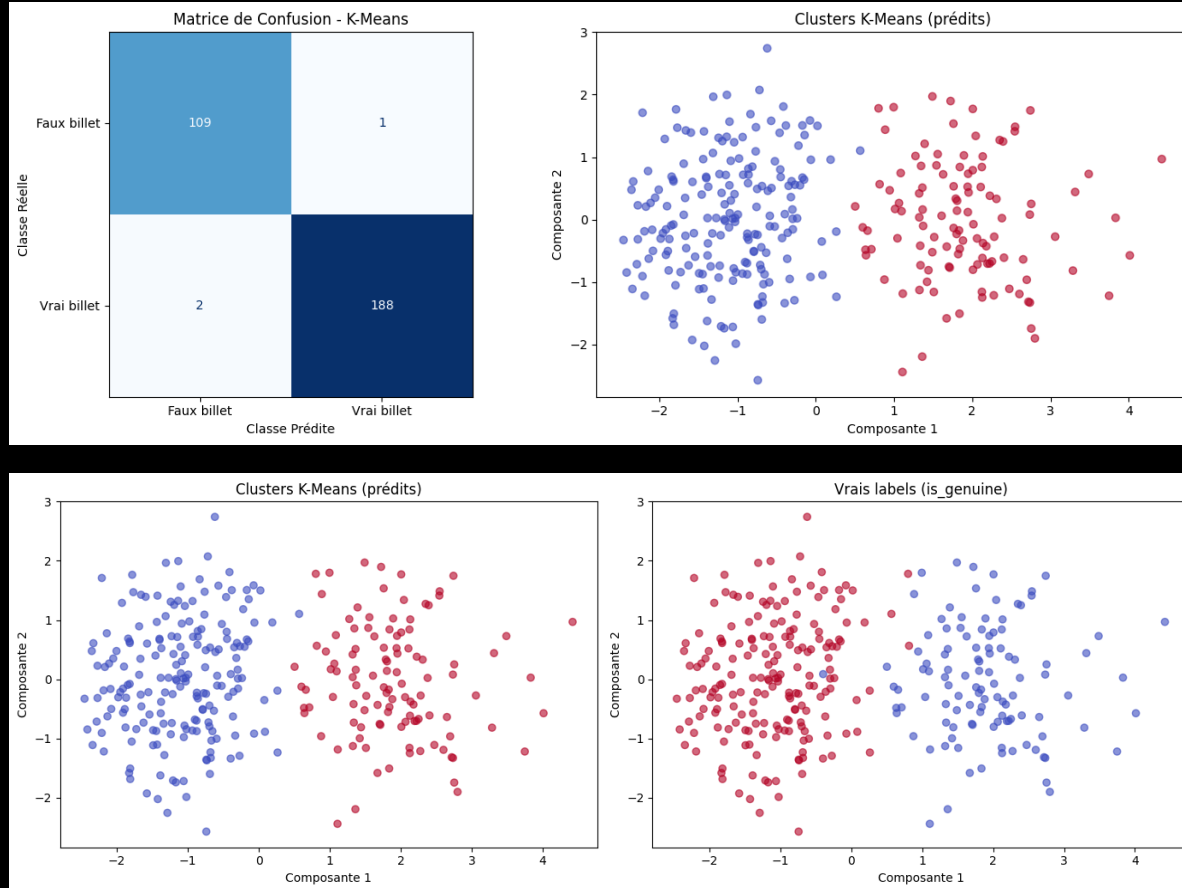
# Modélisation – Random Forest

Que pouvons nous en déduire ?



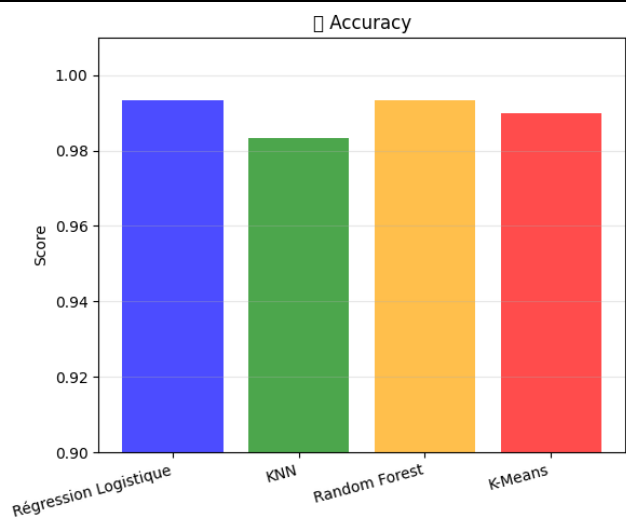
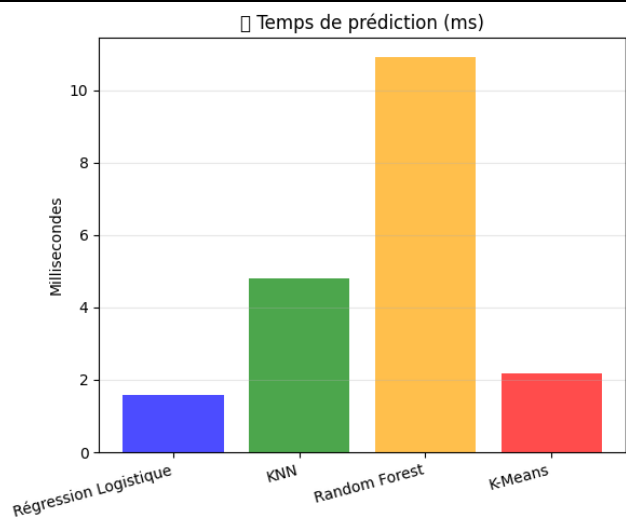
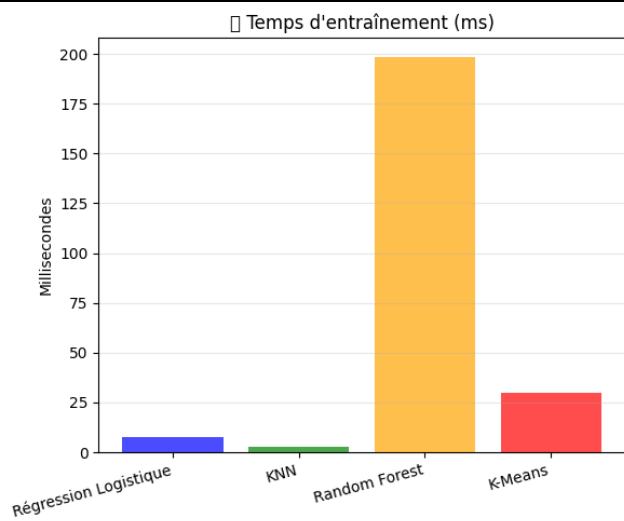
# Modélisation - Kmean

Que pouvons nous en déduire ?



# Comparatif des modèles

| Modèle                | Train (ms) | Predict (ms) | Accuracy |
|-----------------------|------------|--------------|----------|
| Régression Logistique | 12.07      | 2.73         | 0.9933   |
| KNN                   | 5.19       | 8.65         | 0.9833   |
| Random Forest         | 323.99     | 14.12        | 0.9933   |
| K-Means               | 39.88      | 3.36         | 0.9900   |



# Merci

